

Министерство науки и высшего образования Российской Федерации  
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ  
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ  
«Национальный исследовательский университет ИТМО»  
(Университет ИТМО)

Факультет программной инженерии и компьютерной техники

Образовательная программа: Системное и прикладное программное  
обеспечение

Направление подготовки (специальность): 09.04.04 Программная  
инженерия

Наименование практики: Научно-исследовательская работа

## О Т Ч Е Т

о научно-исследовательской работе

Тема задания:

**Исследование архитектурных паттернов серверных платформ  
текстовых многопользовательских онлайн-игр в социальных сетях и  
мессенджерах и подходов к обработке игровых событий, чата и  
внутриигровых транзакций.**

Обучающийся Горинов Даниил Андреевич, № группы Р4116

Руководитель практики от университета:

Белозубов Александр Владимирович, руководитель в ИТМО

Дата 21.06.2026

Санкт-Петербург  
2026

## СОДЕРЖАНИЕ

|                                                                    |           |
|--------------------------------------------------------------------|-----------|
| <b>СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ . . . .</b>            | <b>3</b>  |
| <b>1 Введение . . . . .</b>                                        | <b>4</b>  |
| <b>2 План-график выполнения работы . . . . .</b>                   | <b>5</b>  |
| <b>3 Анализ предметной области и постановка задачи . . . . .</b>   | <b>7</b>  |
| 3.1. Предметная область . . . . .                                  | 7         |
| 3.2. Объект, предмет, цель и задачи исследования . . . . .         | 8         |
| 3.3. Критерии анализа и источниковая база . . . . .                | 8         |
| <b>4 Исследование архитектурных решений и сравнительный анализ</b> | <b>10</b> |
| 4.1. Предварительные архитектурные гипотезы . . . . .              | 10        |
| 4.2. Экспериментальный стенд . . . . .                             | 11        |
| 4.3. Результаты по производительности и масштабируемости . . .     | 13        |
| 4.4. Экспериментальная сравнительная матрица . . . . .             | 15        |
| 4.5. Обоснование архитектурной комбинации . . . . .                | 17        |
| <b>5 Верификация выводов и формирование рекомендаций . . . . .</b> | <b>19</b> |
| 5.1. Матрица отказов и проверка надёжности . . . . .               | 19        |
| 5.2. Трассировка доказательств . . . . .                           | 20        |
| 5.3. Наблюдаемость и проверяемые метрики . . . . .                 | 20        |
| 5.4. Проверка терминологии, источников и ограничений валидности    | 21        |
| 5.5. Практические рекомендации . . . . .                           | 22        |
| <b>6 Заключение . . . . .</b>                                      | <b>23</b> |
| <b>7 Список использованных источников . . . . .</b>                | <b>24</b> |
| <b>ПРИЛОЖЕНИЕ 1 — Репозиторий, команды запуска и структура</b>     |           |
| стенда . . . . .                                                   | 29        |
| <b>ПРИЛОЖЕНИЕ 2 — Raw-сводка экспериментальных артефактов</b>      | <b>30</b> |
| <b>ПРИЛОЖЕНИЕ 3 — Протокол проверки источников и оформления</b>    | <b>31</b> |

# СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ

## Сокращения и условные обозначения

| Сокращение            | Значение                                                                                                                                                                               |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MMORPG                | Massively Multiplayer Online Role-Playing Game; в отчёте рассматривается текстовая многопользовательская ролевая онлайн-игра с серверной обработкой команд, чата и игровых транзакций. |
| API                   | Программный интерфейс взаимодействия клиента, внешней платформы или фонового обработчика с серверной платформой.                                                                       |
| WebSocket             | Двусторонний протокол взаимодействия в реальном времени, определённый RFC 6455 [1].                                                                                                    |
| Outbox                | Таблица исходящих доменных событий, записываемая в одной транзакции с изменением бизнес-состояния.                                                                                     |
| Inbox                 | Механизм фиксации идентификаторов входящих сообщений для защиты от повторной обработки callback и команд.                                                                              |
| Saga                  | Последовательность локальных транзакций с явными состояниями, повторными попытками и компенсациями.                                                                                    |
| Pub/Sub               | Модель публикации и подписки; в Redis Pub/Sub используется для короткоживущих оперативных уведомлений.                                                                                 |
| Streams               | Redis Streams; журнал событий с возможностью чтения истории и восстановления после разрыва соединения.                                                                                 |
| Достоверное состояние | Актуальные данные игровых сущностей, сохранённые в PostgreSQL и используемые для принятия критичных игровых решений.                                                                   |
| DLQ                   | Dead-letter queue, очередь сообщений, которые не удалось обработать после допустимого числа попыток.                                                                                   |
| p50, p95, p99         | 50-й, 95-й и 99-й процентиля задержки обработки команд в экспериментальном стенде.                                                                                                     |

# 1 Введение

Научно-исследовательская работа продолжает анализ, начатый в аналитическом обзоре прошлого семестра «Архитектурные паттерны серверных платформ для текстовых многопользовательских онлайн-игр». В обзорной работе были систематизированы основные паттерны, источники и ограничения предметной области. В данном отчёте обзорное сопоставление дополнено практической проверкой: разработан исследовательский стенд, выполнена серия нагрузочных экспериментов и экспериментов с внедрением отказов, а архитектурные рекомендации уточнены на основании полученных результатов.

Предметная область включает игровые системы, в которых основная нагрузка возникает не от графического рендеринга, а от обработки пользовательских команд, чата, уведомлений в реальном времени, внутриигровой экономики, повторных callback внешних платформ и фоновых процессов. Для таких систем существенны высокая нагрузка, отказоустойчивость, наблюдаемость, контролируемая стоимость эксплуатации и корректность бизнес-состояния.

Цель работы — исследовать и экспериментально сопоставить архитектурные паттерны серверных платформ текстовых многопользовательских онлайн-игр в социальных сетях и мессенджерах, а также обосновать архитектурную комбинацию для обработки игровых событий, чата и внутриигровых транзакций.

Методика работы построена как трассируемая цепочка: критерий сравнения — сценарий стенда — метрика — результат — вывод — рекомендация. Такой порядок позволил не ограничиваться качественными оценками паттернов, а проверять ключевые выводы через источник, реализованный сценарий стенда или экспериментальный артефакт. Оформление отчёта выполнено с учётом требований учебно-методического пособия Университета ИТМО по организации и проведению производственной практики магистрантов: титульный лист, содержание, список сокращений, введение, основная часть, заключение, список источников и приложения; формат A4, Times New Roman 14 пт, полуторный интервал, поля 30/15/20/20 мм [2].

## 2 План-график выполнения работы

Работа выполнялась по плану-графику, приведённому в таблице 2.1. Этапы 2–4 образовали содержательную основу исследования. Этап 5 связан с оформлением письменного отчёта и подготовкой вспомогательных материалов, а этап 6 относится к пост-отчётному оформлению отзыва руководителя в модуле «Практика».

Таблица 2.1 – План-график выполнения НИР

| Этап | Срок    | Название этапа                                                     | Задание                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|------|---------|--------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | 1 день  | Инструктаж обучающегося                                            | Инструктаж обучающегося по ознакомлению с требованиями охраны труда, техники безопасности, пожарной безопасности, а также правилами внутреннего трудового распорядка.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| 2    | 44 дня  | Анализ предметной области и постановка задачи исследования         | Провести анализ предметной области серверных платформ текстовых многопользовательских онлайн-игр в социальных сетях и мессенджерах. Определить объект, предмет, цель и задачи исследования. Рассмотреть особенности таких систем: высокую нагрузку, требования к отказоустойчивости, обработку игровых событий в реальном времени, обмен сообщениями в чате и выполнение внутриигровых транзакций. Выделить ключевые критерии сравнения архитектурных решений: масштабируемость, надёжность, производительность, сложность внедрения и стоимость эксплуатации. Подобрать и систематизировать научные и промышленные источники по теме. Результат: постановка задачи исследования, перечень критериев анализа и обзор источников по теме работы.                                                                                               |
| 3    | 60 дней | Исследование архитектурных решений и сравнительный анализ подходов | Провести исследование архитектурных паттернов и инфраструктурных решений, применяемых при построении серверных платформ текстовых онлайн-игр. Рассмотреть и сопоставить подходы к обработке игровых событий, сообщений чата, взаимодействия в реальном времени и внутриигровых транзакций. Выполнить анализ таких решений, как событийно-ориентированная архитектура, Transactional Outbox/Inbox, оркестрация и хореография Saga, WebSocket-инфраструктура, брокеры сообщений, кэширование и использование PostgreSQL и Redis в составе игровой платформы. Сравнить указанные подходы по ранее выбранным критериям и определить их преимущества, ограничения и область применения. Результат: сравнительный анализ архитектурных решений и обоснование наиболее подходящей архитектурной комбинации для серверной платформы текстовой MMORPG. |
| 4    | 16 дней | Верификация выводов и формирование рекомендаций                    | Проверить логическую непротиворечивость полученных результатов исследования и соответствие сделанных выводов поставленным задачам. Уточнить используемую терминологию, проверить корректность ссылок на источники и полноту рассмотрения исследуемых архитектурных решений. На основании проведённого анализа сформулировать итоговые выводы и практические рекомендации по проектированию серверной платформы текстовой многопользовательской онлайн-игры, включая рекомендации по обработке игровых событий, чата и внутриигровых транзакций. Результат: верифицированные выводы исследования и набор практических рекомендаций по проектированию архитектуры серверной платформы.                                                                                                                                                          |

## Продолжение таблицы 2.1 — План-график выполнения НИР

| Этап | Срок                      | Название этапа                         | Задание                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|------|---------------------------|----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 5    | 02.03.2026–<br>21.06.2026 | Оформление отчётных документов         | <p>Подробное описание задач по этапам, результаты размещаются в приложениях. Оформление выполняется в соответствии с методическим пособием:</p> <p><a href="https://books.ifmo.ru/file/pdf/2622.pdf">https://books.ifmo.ru/file/pdf/2622.pdf</a>. Структура документа: титульный лист, введение, основная часть, заключение, приложения. В основной части подробно описывается выполнение задач этапов 2–4, в приложениях размещаются результаты данных этапов. Отчёт необходимо загрузить в модуле «Практика» как «письменный отчёт».</p> |
| 6    | 22.06.2026–<br>27.06.2026 | Получение отзыва руководителя практики | Получение отзыва с оценкой у руководителя практики в модуле «Практика».                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |

### 3 Анализ предметной области и постановка задачи

#### 3.1 Предметная область

На первом содержательном этапе была использована преемственность с аналитическим обзором, подготовленным в предыдущем семестре. В нём уже были выделены основные классы архитектурных решений для текстовых многопользовательских онлайн-игр: Outbox/Inbox, Saga, WebSocket-доставка, кэширование, PostgreSQL и Redis. В текущей НИР требовалось уточнить не только перечень паттернов, но и то, какие свойства этих паттернов действительно проверяются в условиях, близких к серверной платформе текстовой MMORPG.

В работе текстовая MMORPG рассматривается как многопользовательская система, в которой пользователь взаимодействует с игровым миром через команды, сообщения, уведомления, кнопки интерфейса социальной сети или мессенджера. В отличие от графических онлайн-игр, такая система не требует постоянной синхронизации трёхмерной сцены, но предъявляет высокие требования к серверной логике: экономические операции должны быть транзакционными, чат должен доставляться с малой задержкой, состояние игрока должно восстанавливаться после сбоя, а повторный callback внешней платформы не должен приводить к повторному начислению валюты или предмета.

После повторного анализа источников сценарии платформы были разделены по критичности состояния и требованиям к доставке. Исследования серверных архитектур MMOG показывают, что серверная часть многопользовательской игры должна совмещать долговременное хранение состояния, сетевую доставку, масштабирование и контроль согласованности [3, 4]. Работы по транзакционным моделям и требованиям согласованности для MMOG подчёркивают, что действия игроков конкурируют за игровые ресурсы, предметы, валюту и состояние мира, поэтому простая обработка сообщений без модели согласованности недостаточна [5—8]. С учётом этого выделены классы сценариев серверной платформы, приведённые в таблице 3.1.

Таблица 3.1 – Классы сценариев серверной платформы текстовой MMORPG

| Класс сценариев                      | Типичные операции                                                               | Архитектурное требование                                                                                            |
|--------------------------------------|---------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Игровые команды                      | перемещение по локациям, выполнение задания, получение награды                  | транзакционная фиксация состояния, идемпотентность повторной команды, аудит результата                              |
| Чат и сообщения                      | сообщения комнаты, личные сообщения, системные уведомления                      | доставка в реальном времени, рассылка активным клиентам, отделение короткоживущих сообщений от критичного состояния |
| Игровая экономика и инвентарь игрока | списание валюты, покупка предмета, выдача награды, обмен                        | строгие правила изменения состояния, уникальные ключи, локальные транзакции, Saga для цепочек с компенсациями       |
| Интеграции и callback                | ответы внешней платформы, повторные webhook, отложенные задания                 | Inbox, correlation id, устойчивость к повторной доставке                                                            |
| Состояние в реальном времени         | присутствие игрока онлайн, обновления интерфейса, повторное подключение клиента | WebSocket, heartbeat, backpressure, повторное чтение событий или snapshot после разрыва соединения                  |
| Фоновые процессы                     | начисление периодических наград, события мира, обработка очередей               | Outbox, пулы фоновых обработчиков, метрики lag/retry/DLQ, безопасная остановка и повторный запуск                   |

### 3.2 Объект, предмет, цель и задачи исследования

Объект исследования — серверные платформы текстовых MMORPG, рассчитанные на взаимодействие пользователей через социальные сети, мессенджеры и web-клиенты. Предмет исследования — архитектурные паттерны и инфраструктурные решения, обеспечивающие обработку игровых событий, чата, взаимодействия в реальном времени и внутриигровых транзакций.

Цель исследования — выбрать архитектурную комбинацию, которая обеспечивает корректность критичного состояния, поддерживает доставку в реальном времени и остаётся реализуемой при росте нагрузки. Для достижения цели было недостаточно ограничиться обзорным перечислением архитектурных решений; требовалось связать каждое решение с проверяемым сценарием стенда, метрикой и наблюдаемым результатом. Поэтому в работе сразу зафиксированы проверяемые сценарии, характерные для текстовой MMORPG: повторная доставка callback, сбой после публикации события, потеря короткоживущего сообщения, повторное подключение клиента, устаревшее чтение кэша и частичная игровая экономическая операция.

### 3.3 Критерии анализа и источниковая база

Критерии сравнения взяты из задания на практику и уточнены для экспериментальной проверки. В таблице 3.2 каждому критерию сопоставле-



ны проверяемые сценарии и метрики стенда. Такой способ нужен для того, чтобы итоговые рекомендации были связаны с наблюдаемыми данными, а не только с общими характеристиками технологий.

Таблица 3.2 – Критерии сравнения и способ их проверки

| Критерий               | Содержание критерия                                                                                                                  | Сценарий стенда                                                                                                             | Метрики                                                                                         |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Масштабируемость       | способность увеличивать число команд, сообщений, фоновых обработчиков и подключений                                                  | серии 100/500/1000 для синхронной транзакции, Outbox-транзакции и чатовой рассылки                                          | пропускная способность, доля ошибок, длина журнала событий                                      |
| Надёжность             | устойчивость к повторам, сбою процесса публикации, потере короткоживущих сообщений, повторному подключению и устаревшему чтению кэша | повторный callback, сбой после публикации, потеря Pub/Sub-сообщения, WebSocket-восстановление, Saga, устаревшее чтение кэша | дубли, остаток Outbox, полученные события, признак устаревшего значения, завершение/компенсация |
| Производительность     | влияние решения на задержку и пропускную способность                                                                                 | серии задержки для синхронной транзакции, Outbox-транзакции и чатовой рассылки                                              | p50, p95, p99, пропускная способность                                                           |
| Сложность внедрения    | число компонентов, состояний, контрактов и путей фоновой обработки                                                                   | реализованные компоненты стенда                                                                                             | число компонентов, состояний и контрактов; состояния Saga                                       |
| Стоимость эксплуатации | инфраструктурная и операционная цена решения                                                                                         | зависимости с собственным состоянием, сохраняемая история, очередь необработанных событий                                   | Redis, PostgreSQL, фоновый обработчик, длина журнала событий, попытки публикации                |
| Наблюдаемость          | возможность доказать или опровергнуть архитектурный вывод метрикой                                                                   | /metrics/summary, CSV/JSON                                                                                                  | возраст очереди, lag, дубли, длительность Saga, устаревшее чтение кэша                          |

Источники использовались по принципу применимости к конкретному утверждению. Рецензируемые работы применялись для анализа долговременного хранения состояния, согласованности и транзакционных моделей MMOG [3—8]. Книги и паттерны распределённых систем использовались для терминов и интеграционных решений [9—13]. RFC и официальные документации применялись для свойств конкретных технологий [1, 14—17]. Индустриальные материалы Microsoft, Microservices.io, Event-Driven.io, InfoQ, Ably и WebSocket.org использовались для современных эксплуатационных практик [18—24]. Русскоязычные публикации по WebSocket/Socket.IO применялись как дополнительная сверка терминологии приложений реального времени [25, 26].

## 4 Исследование архитектурных решений и сравнительный анализ

### 4.1 Предварительные архитектурные гипотезы

После постановки задачи рассматриваемые решения были распределены по ролям, чтобы не сравнивать их как взаимоисключающие технологии. Данные, хранимые в PostgreSQL, рассматривались как достоверное состояние игровых сущностей: игровой экономики, прогресса игрока, инвентаря игрока и аудита игровых операций. Transactional Outbox использовался как способ атомарно связать изменение состояния и исходящее событие; Inbox — как защита от повторной доставки; Redis Streams — как сохраняемый журнал событий для повторного чтения; Redis Pub/Sub — как короткоживущий канал оперативной рассылки активным клиентам; WebSocket-шлюз — как интерфейс клиента для обмена в реальном времени; Saga — как механизм координации долгих игровых экономических цепочек; cache-aside — как ускорение чтения некритичных данных.

Эти роли согласуются с источниками: Outbox устраняет проблему двойной записи, но допускает повторную публикацию при сбое процесса публикации между отправкой события и отметкой доставки [19, 20]; Pub/Sub не хранит историю для позднего подписчика [15]; Redis Streams предоставляет журнал для чтения истории [14]; WebSocket является транспортом обмена в реальном времени, но не заменяет долговременное хранение игрового состояния или журнал событий [1, 23]; Saga применяется для последовательности локальных транзакций с компенсациями [21, 22].

Предварительная гипотеза состояла в том, что серверная платформа текстовой MMORPG должна строиться как комбинация перечисленных решений, а не как выбор одного универсального брокера, одной базы данных или одного протокола реального времени. Эта гипотеза затем проверялась на стенде. Архитектурная схема проверяемой комбинации приведена на рисунке 4.1.

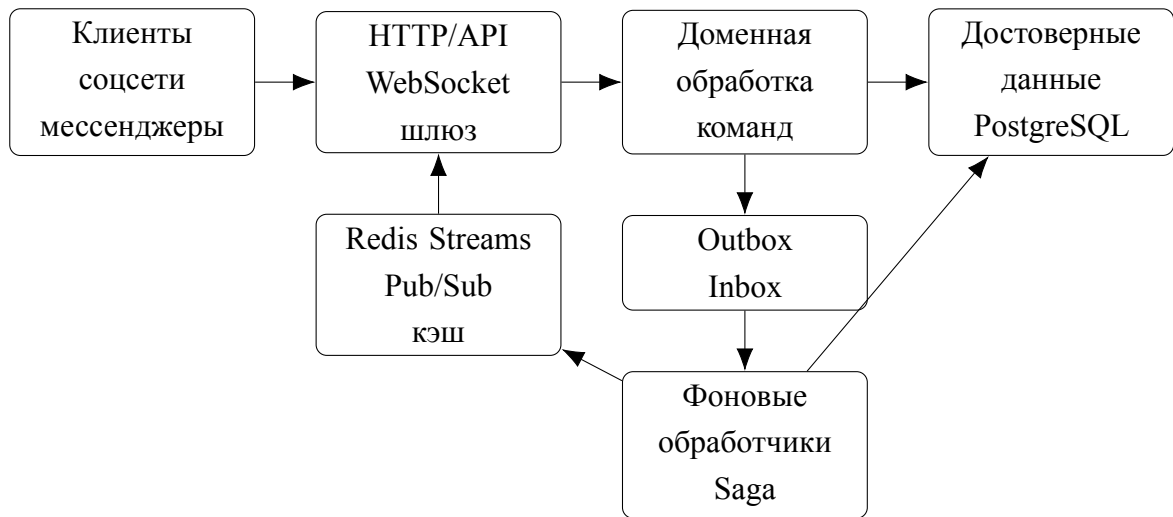


Рисунок 4.1 – Архитектурная комбинация, проверенная на исследовательском стенде

## 4.2 Экспериментальный стенд

Для проверки гипотезы был разработан самостоятельный стенд, опубликованный в репозитории [27]. Серверная часть реализована на Go: стенд должен был быстро поднимать HTTP API, фоновые обработчики и точку WebSocket-подключения без лишней инфраструктуры. Для сценариев нагрузки, внедрения отказов и построения графиков использовался Python, поскольку в нём удобно собирать CSV/JSON-артефакты и строить повторяемые визуализации. PostgreSQL и Redis запускались вместе с серверной частью и фоновыми обработчиками через Docker Compose, чтобы проверка не зависела от внешних сервисов.

Стенд моделирует минимальную серверную платформу текстовой игры: команды игрока изменяют баланс золота; Inbox фиксирует идентификаторы входящих сообщений; Outbox сохраняет исходящие события; фоновый обработчик публикует события в Redis Streams и Pub/Sub; WebSocket-клиент проверяет повторное чтение событий; обработчик Saga завершает или компенсирует покупку; сценарий cache-aside демонстрирует устаревшее чтение.

Первой технической сложностью стал выбор границы между локальной транзакцией PostgreSQL и асинхронной публикацией события. Если публиковать событие напрямую из обработчика команды, появляется риск двойной записи: состояние уже изменено, а событие не доставлено, либо наоборот. Поэтому публикация была вынесена в Outbox, а сценарий

`publish_no_ack` проверен отдельно: процесс публикации успевает отправить событие, но не успевает отметить его доставленным. Этот сценарий был важен для отчёта, потому что он показывает не абстрактный риск, а конкретное место, где без идемпотентного потребителя возникает дубль.

Вторую сложность вызвал выбор модели доставки в реальном времени. Redis Pub/Sub удобен для оперативной рассылки активным клиентам, но не хранит историю для клиента, который подключился позднее. Redis Streams, наоборот, сохраняет события и позволяет выполнить повторное чтение, но требует контроля длины журнала и политики хранения. Поэтому в стенде реализованы оба механизма: Pub/Sub используется как быстрый короткоживущий канал, а Streams — как журнал для восстановления после повторного подключения. Для WebSocket-проверки добавлен отдельный клиент, который подключается с начальным `last=0-0` и подтверждает, что события можно получить повторно из журнала.

Отдельно реализованы Saga-сценарий и cache-aside сценарий. Saga понадобилась для проверки долгой игровой экономической операции, где возможны два корректных исхода: завершение и компенсация. Cache-aside был включён в стенд не как рекомендация для критичных игровых операций, а как контролируемая демонстрация риска устаревшего чтения: кэш может вернуть старое значение, если актуальное состояние в PostgreSQL уже изменилось.

Стенд не претендует на моделирование всего промышленного кластера MMORPG. Его назначение — воспроизвести причинные свойства архитектурных паттернов в контролируемой среде. Поэтому абсолютные значения задержки и пропускной способности трактуются как локальные экспериментальные значения, а не как универсальный эталон производительности. Достоверность выводов обеспечивается повторяемостью сценариев, фиксированным seed 4116, проверкой инвариантов и сохранением CSV/JSON-артефактов.

Таблица 4.1 – Список сценариев экспериментального стенда

| Сценарий                 | Что проверяется                                             | Метрики                                                 | Артефакты                       |
|--------------------------|-------------------------------------------------------------|---------------------------------------------------------|---------------------------------|
| Синхронная транзакция    | базовая локальная транзакция PostgreSQL                     | p50/p95/p99, пропускная способность, доля ошибок        | scenario_runs.csv, summary.json |
| Outbox-транзакция        | запись Outbox и асинхронная публикация фоновым обработчиком | p50/p95/p99, очередь Outbox, попытки публикации         | summary.json                    |
| Чатовая рассылка         | запись и доставка чатовых сообщений                         | задержка, пропускная способность, длина журнала событий | scenario_runs.csv               |
| Повторный callback       | повторная доставка одной команды                            | дубли, итоговый баланс золота                           | summary.json                    |
| Сбой после публикации    | публикация события без отметки доставки                     | дубли Redis, остаток очереди Outbox                     | summary.json                    |
| Потеря Pub/Sub-сообщения | потеря сообщения для позднего подписчика                    | признак доставки                                        | summary.json                    |
| Устаревшее чтение кэша   | чтение старого значения через cache-aside                   | значение в кэше, актуальное значение                    | summary.json                    |
| Saga                     | успешная и компенсированная покупка                         | завершение, компенсация, длительность                   | summary.json                    |
| WebSocket-восстановление | восстановление событий после повторного подключения         | число повторно полученных событий                       | summary.json                    |

### 4.3 Результаты по производительности и масштабируемости

После разработки прототипа было развёрнуто тестовое окружение и выполнена нагрузочная часть эксперимента. Для сценариев «синхронная транзакция», «Outbox-транзакция» и «чатовая рассылка» использованы размеры нагрузки 100, 500 и 1000 операций; каждый размер выполнен 5 раз. Итоговые агрегированные значения приведены в таблице 4.2. Ошибки во всех строках равны нулю, что важно для интерпретации пропускной способности: серия не скрывает падения запросов за счёт уменьшения числа успешных операций.

Таблица 4.2 – Агрегированные результаты нагрузочных серий

| Сценарий              | Нагрузка | Повт. | p50, мс | p95, мс | p99, мс | Пропускная способность, запросов/с | Доля ошибок |
|-----------------------|----------|-------|---------|---------|---------|------------------------------------|-------------|
| Чатовая рассылка      | 100      | 5     | 0,546   | 1,001   | 1,508   | 514,16                             | 0           |
| Чатовая рассылка      | 500      | 5     | 0,371   | 0,697   | 1,055   | 673,89                             | 0           |
| Чатовая рассылка      | 1000     | 5     | 0,430   | 0,875   | 1,720   | 599,37                             | 0           |
| Outbox-транзакция     | 100      | 5     | 0,613   | 0,937   | 1,223   | 584,08                             | 0           |
| Outbox-транзакция     | 500      | 5     | 0,606   | 1,059   | 1,734   | 564,74                             | 0           |
| Outbox-транзакция     | 1000     | 5     | 0,760   | 1,203   | 1,750   | 465,78                             | 0           |
| Синхронная транзакция | 100      | 5     | 0,571   | 0,956   | 1,369   | 584,74                             | 0           |
| Синхронная транзакция | 500      | 5     | 0,529   | 0,877   | 1,243   | 583,86                             | 0           |
| Синхронная транзакция | 1000     | 5     | 0,641   | 1,099   | 1,748   | 508,99                             | 0           |

На рисунках 4.2 и 4.3 сопоставлены тенденции p95 и пропускной способности при росте размера серии. Для локального стенда при нагрузке 1000 операций p95 составил 0,875 мс для чатовой рассылки, 1,099 мс для синхронной транзакции и 1,203 мс для Outbox-транзакции. Это не означает, что промышленная система будет иметь такие же задержки; вывод состоит в другом: добавление Outbox в командный путь на данном стенде не привело к неконтролируемому росту задержки, а очередь Outbox после обработки была закрыта. Следовательно, Outbox допустим для критичных доменных событий при обязательном мониторинге очереди необработанных событий и повторных попыток.

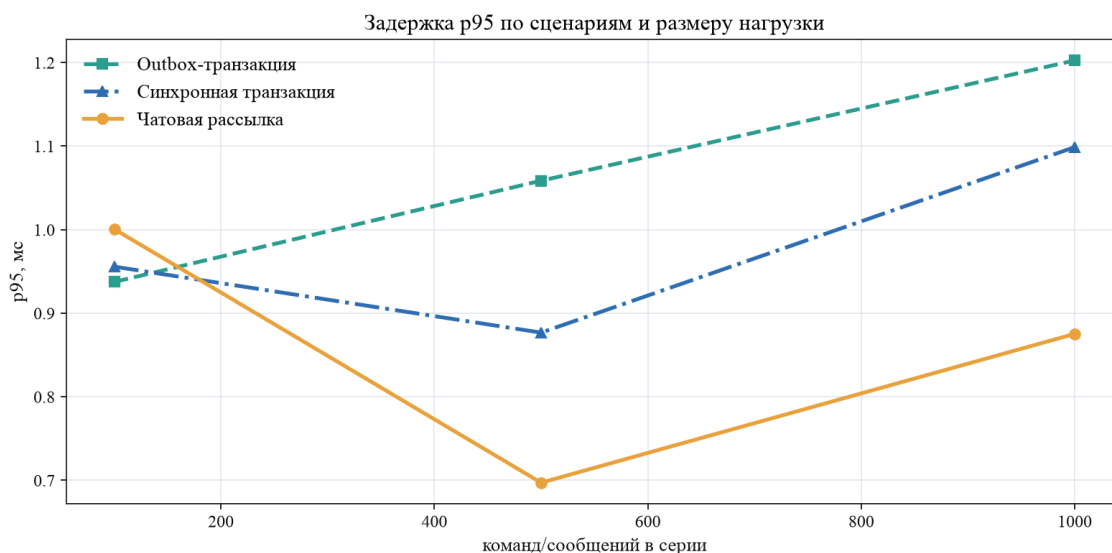


Рисунок 4.2 – Сравнение p95 задержки по сценариям и размерам нагрузки

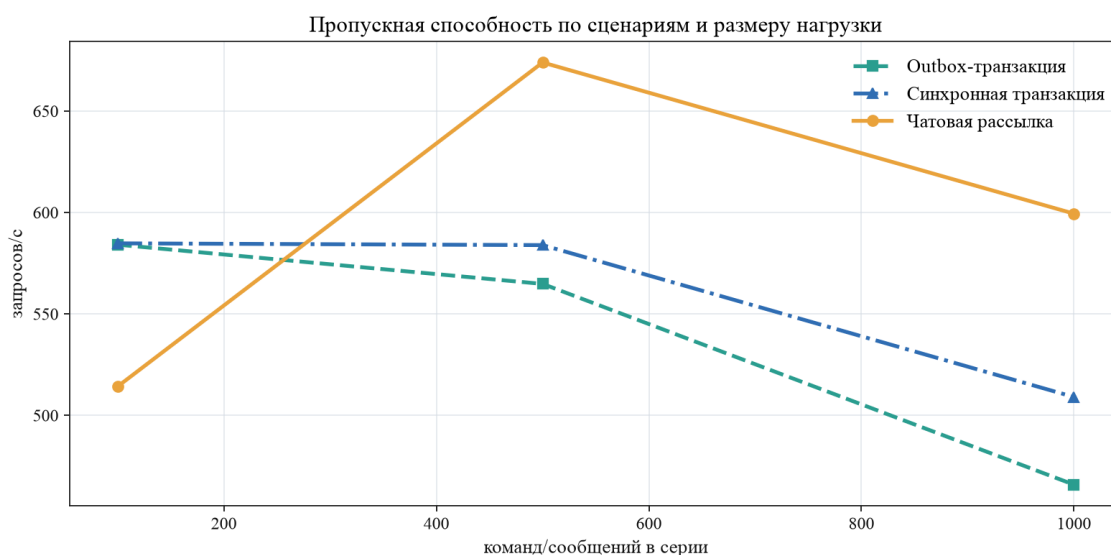


Рисунок 4.3 – Сравнение пропускной способности по сценариям и размерам нагрузки

#### 4.4 Экспериментальная сравнительная матрица

Итоговое сравнение построено по результатам `comparison_matrix.csv`. Оценка от 1 до 5 отражает пригодность решения для задач текстовой MMORPG в проверенных условиях. Для производительности и масштабируемости использованы результаты нагрузочных серий; для надёжности — сценарии внедрения отказов; для сложности внедрения, стоимости эксплуатации и наблюдаемости — реализованные компоненты стенда, число состояний, зависимости с собственным состоянием и доступные метрики.

Таблица 4.3 – Экспериментальная матрица сравнения архитектурных решений

| Решение                          | М | Н | П | СВ | СЭ | Набл. | Итог | Экспериментальное основание                            | Вывод                                                                                                    |
|----------------------------------|---|---|---|----|----|-------|------|--------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| Синхронная транзакция PostgreSQL | 4 | 4 | 5 | 4  | 4  | 3     | 4,00 | p95=1,099 мс; пропускная способность=508,99 запросов/с | подходит для изменения критичного состояния, но не решает рассылку событий                               |
| Transactional Outbox             | 4 | 5 | 5 | 3  | 3  | 5     | 4,17 | p95=1,203 мс; остаток=0                                | обеспечивает восстановимую публикацию после фиксации транзакции при фоновой обработке и метриках очереди |
| Inbox и идиempотентность         | 5 | 5 | 5 | 4  | 5  | 4     | 4,67 | 5 повторов дали 4 дубля и один бизнес-результат        | предотвращает повторный бизнес-эффект при повторной доставке                                             |

## Продолжение таблицы 4.3 — Экспериментальная матрица сравнения архитектурных решений

| Решение          | М | Н | П | СВ | СЭ | Набл. | Итог | Экспериментальное основание                            | Вывод                                                                                        |
|------------------|---|---|---|----|----|-------|------|--------------------------------------------------------|----------------------------------------------------------------------------------------------|
| Redis Streams    | 5 | 5 | 4 | 3  | 3  | 5     | 4,17 | stream_len=8005; повторно получено 3 события           | подходит для повторного чтения и контролируемой доставки, требует политики хранения          |
| Redis Pub/Sub    | 5 | 2 | 5 | 5  | 4  | 2     | 3,83 | поздний подписчик: false                               | пригоден для оперативной рассылки, но не для гарантированной доставки критичного состояния   |
| WebSocket-шлюз   | 4 | 4 | 4 | 3  | 3  | 4     | 3,67 | WebSocket-восстановление=3                             | нужен для доставки в реальном времени, восстановление должно опираться на сохраняемый журнал |
| Saga-оркестрация | 3 | 5 | 3 | 2  | 2  | 5     | 3,33 | завершено=1; компенсировано=1; p95=23,325 мс           | подходит для долгих игровых экономических цепочек с явными компенсациями                     |
| Cache-aside      | 5 | 2 | 5 | 4  | 4  | 3     | 3,83 | кэш=100; актуально=150; устарело=true                  | ускоряет чтение, но не может быть основанием критичных игровых решений                       |
| Чатовая рассылка | 4 | 4 | 5 | 4  | 4  | 4     | 4,17 | p95=0,875 мс; пропускная способность=599,37 запросов/с | подходит для массовых сообщений при разделении Pub/Sub и сохраняемого журнала                |

Обозначения столбцов таблицы 4.3: М — масштабируемость, Н — надёжность, П — производительность, СВ — сложность внедрения, СЭ — стоимость эксплуатации, Набл. — наблюдаемость. Наибольший итоговый балл получил механизм Inbox и идемпотентности, что отражает критичность защиты от повторного бизнес-эффекта в системах с внешними callback. Чатовая рассылка получила высокий итоговый балл за производительность и масштабируемость, а Outbox уступил ей по итоговой оценке из-за большей сложности и эксплуатационной цены, но остаётся необходимым для восстановимой публикации событий.

Рисунок 4.4 дополняет таблицу 4.3: он показывает не все строки матрицы, а профиль ключевых решений, определяющих итоговую архитектурную комбинацию. Такая форма позволяет увидеть инженерный компромисс: Inbox и Outbox усиливают надёжность и наблюдаемость, Redis Pub/Sub даёт высокую скорость оперативной рассылки при слабой надёжности для поздних подписчиков, Saga имеет выраженную цену сложности, а cache-aside требует ограничения области применения из-за риска устаревшего чтения.



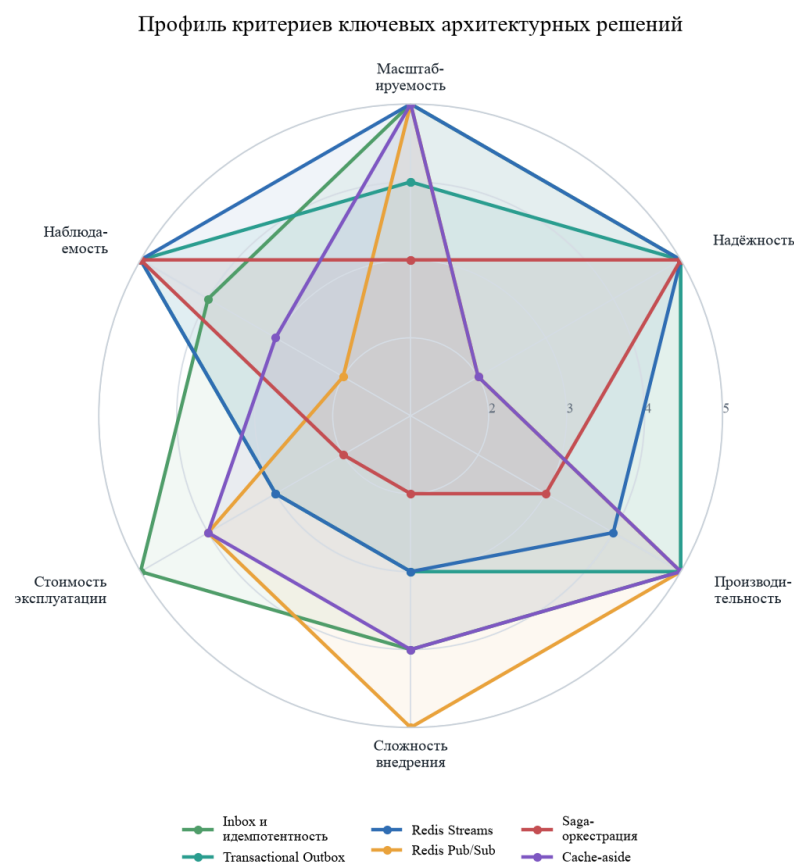


Рисунок 4.4 – Профиль критериев ключевых архитектурных решений по экспериментальной матрице

## 4.5 Обоснование архитектурной комбинации

На основании источников и эксперимента обоснована следующая архитектурная комбинация.

Достоверное состояние игровой экономики, прогресса игрока, инвентаря игрока и аудита игровых операций должно храниться в PostgreSQL. Это следует из требований к локальным ACID-транзакциям, уникальным ограничениям и восстановлению состояния [16, 17]. Экспериментальная серия показала, что локальная транзакционная обработка обеспечивает устойчивую базовую задержку и нулевую долю ошибок на стенде, но сама по себе не решает доставку событий внешним потребителям.

Transactional Outbox следует использовать для доменных событий,

связанных с изменением состояния. Эксперимент `publish_no_ack` воспроизвёл транспортный дубль: событие было опубликовано, но отметка доставки не была зафиксирована, после чего фоновый обработчик повторил публикацию. Это подтверждает известное ограничение Outbox: паттерн устраняет двойную запись, но не отменяет требования к идемпотентным потребителям [11, 20]. Поэтому Outbox должен применяться вместе с Inbox, уникальными ключами и дедупликацией на стороне потребителя.

Redis Pub/Sub и Redis Streams должны использоваться совместно, но для разных гарантий. Pub/Sub подходит для оперативной рассылки активным WebSocket-шлюзам, поскольку имеет малую задержку и простую модель подписки; при этом сценарий `pubsub_loss` подтвердил, что поздний подписчик не получает уже опубликованное сообщение. Redis Streams, напротив, хранит события и обеспечивает повторное чтение: WebSocket-проверка получила 3 события после подключения с начальным `last=0-0`. Следовательно, критичное состояние восстанавливается не из Pub/Sub, а из данных PostgreSQL или сохраняемого журнала событий.

Saga-оркестрация оправдана для долгих игровых экономических цепочек, где несколько локальных транзакций должны завершиться согласованным финалом. На стенде были получены оба наблюдаемых исхода: одна операция завершилась, одна операция была компенсирована. При этом Saga имеет более высокую сложность внедрения и эксплуатации, поэтому не должна заменять простые локальные транзакции для обычных команд.

Cache-aside следует применять только для справочных и некритичных игровых данных. Сценарий `cache_stale` зафиксировал расхождение между значением в кэше 100 и актуальным значением 150; поэтому решения о списании валюты, выдаче предметов и изменении прав владения должны приниматься по актуальному состоянию, сохранённому в PostgreSQL. Такая граница согласуется с ограничениями кэширующих стратегий: ускорение чтения не заменяет долговременное хранение состояния и контроль устаревания данных [28].

## 5 Верификация выводов и формирование рекомендаций

### 5.1 Матрица отказов и проверка надёжности

После основных нагрузочных серий были выполнены проверки с внедрением отказов. Выбраны отказы, которые соответствуют типичным рискам серверной платформы текстовой MMORPG: повторная доставка callback, сбой процесса публикации после отправки события, потеря краткоживущего сообщения, повторное подключение клиента, устаревшее чтение кэша и частичная Saga. Результаты приведены в таблице 5.1.

Таблица 5.1 – Результаты сценариев с внедрением отказов

| Сценарий                 | Результат                                    | Расшифровка результата                                                                           | Вывод                                                                    |
|--------------------------|----------------------------------------------|--------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| Повторный callback       | 5 запросов; 4 дубля; итоговый баланс 7       | повторная доставка одной команды не изменила игровое состояние повторно                          | обязателен Inbox и idempotency key                                       |
| Сбой после публикации    | дубли Redis: 1; остаток Outbox: 0            | транспортный дубль возможен, но очередь Outbox закрылась после повторной обработки               | нужны идемпотентные потребители и мониторинг Outbox                      |
| Потеря Pub/Sub-сообщения | delivered=false                              | поздний подписчик не получает историю уже опубликованных сообщений                               | Pub/Sub нельзя использовать как гарантированный журнал критичных событий |
| WebSocket-восстановление | received=3                                   | повторное подключение восстановило события из журнала Redis Streams                              | нужен маркер последнего полученного события и сохраняемый журнал         |
| Устаревшее чтение кэша   | кэш=100; актуально=150; устарело=true        | cache-aside может вернуть значение, которое уже отличается от актуального состояния в PostgreSQL | кэш не должен использоваться для критичных игровых решений               |
| Исходы Saga              | завершено=1; компенсировано=1; p95=23,325 мс | оба исхода долгой операции наблюдаемы и проверяемы                                               | Saga требует явных состояний и компенсаций                               |

Содержательный результат проверок с внедрением отказов состоит в том, что каждый проверяемый отказ связан с проектным решением. Повторная доставка callback требует Inbox; `publish_no_ack` требует идемпотентных потребителей; потеря Pub/Sub-сообщения требует сохраняемого журнала или восстановления из PostgreSQL; повторное подключение WebSocket требует маркера последнего полученного события; устаревшее чтение кэша требует запрета на использование кэша для критичных игровых решений; Saga требует явных состояний и компенсаций.

## 5.2 Трассировка доказательств

Чтобы не оставить разрыв между экспериментом и рекомендациями, сформирована трассировка доказательств: каждому критерию сопоставлены сценарий, метрика, артефакт, результат и вывод. Сводная версия приведена в таблице 5.2; полная JSON-форма сохранена в `results/latest/evidence_trace.json`.

Таблица 5.2 – Трассировка доказательств от критерия к выводу

| Критерий               | Сценарий                                                                                                 | Метрика                                                                               | Артефакт                                                       | Вывод                                                                                                 |
|------------------------|----------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|----------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| Масштабируемость       | серии 100/500/1000 для синхронной транзакции, Outbox-транзакции и чатовой рассылки                       | пропускная способность и доля ошибок                                                  | <code>scenario_runs.csv</code> , <code>summary.json</code>     | масштабируемые каналы нужны прежде всего для рассылки и фоновой обработки событий                     |
| Надёжность             | повторный callback, сбой после публикации, потеря Pub/Sub-сообщения, WebSocket-восстановление, кэш, Saga | дубли, остаток очереди, признак доставки, устаревшее значение, завершение/компенсация | <code>summary.json</code> , таблица отказов                    | надёжность достигается комбинацией Outbox, Inbox, Streams, Saga и достоверного состояния в PostgreSQL |
| Производительность     | серии задержки и пропускной способности                                                                  | p50, p95, p99, пропускная способность                                                 | <code>scenario_runs.csv</code> , таблица нагрузочных серий     | накладные расходы Outbox приемлемы только при контроле очереди и повторных попыток                    |
| Сложность внедрения    | реализованные компоненты и автоматы состояний                                                            | число компонентов, состояний и контрактов                                             | <code>comparison_matrix.csv</code>                             | сложные паттерны применяются только там, где локальная транзакция не закрывает риск                   |
| Стоимость эксплуатации | зависимости с собственным состоянием и сохраняемая история                                               | длина журнала событий, попытки публикации, PostgreSQL, Redis, фоновый обработчик      | <code>comparison_matrix.csv</code> , <code>summary.json</code> | стоимость определяется политикой хранения Redis Streams/Outbox и наблюдаемостью фоновых обработчиков  |
| Наблюдаемость          | <code>/metrics/summary</code> и артефакты                                                                | возраст очереди Outbox, дубли, длительность Saga, устаревшее чтение кэша              | <code>summary.json</code> , <code>evidence_trace.json</code>   | рекомендация считается проверенной только при наличии наблюдаемой метрики                             |

## 5.3 Наблюдаемость и проверяемые метрики

При проверке стенда отдельно выделена наблюдаемость. Для архитектуры серверной платформы текстовой MMORPG это не вспомогательное свойство, а условие управляемости. Если система не измеряет возраст очереди Outbox, число повторных публикаций, отставание журнала событий, длительность Saga и устаревшие значения кэша, то эксплуатационная команда

не сможет отличить корректную повторную доставку от потери события или зависшего фонового обработчика. В стенде эта связь отражена через метод `/metrics/summary`; ключевые метрики приведены в таблице 5.3.

Таблица 5.3 – Метрики наблюдаемости, использованные в исследовании

| Метрика                                        | Значение в запуске | Что подтверждает                                               |
|------------------------------------------------|--------------------|----------------------------------------------------------------|
| <code>inbox_count</code>                       | 16003              | команды и callback фиксируются для дедупликации                |
| <code>outbox_published</code>                  | 8004               | фоновый обработчик реально публиковал исходящие события        |
| <code>outbox_attempts_total</code>             | 8005               | повторная попытка после <code>publish_no_ack</code> наблюдаема |
| <code>outbox_pending</code>                    | 0                  | очередь Outbox закрыта после завершения эксперимента           |
| <code>outbox_oldest_pending_ms</code>          | 0                  | незакрытых старых событий нет                                  |
| <code>redis_stream_len</code>                  | 8005               | Redis Streams содержит историю для повторного чтения           |
| <code>redis_duplicate_events</code>            | 1                  | транспортный дубль воспроизведён и измерен                     |
| <code>chat_messages</code>                     | 8000               | чатовая нагрузка обработана полностью                          |
| <code>chat_stream_len</code>                   | 8000               | чатовые события сохранены в журнале Redis Streams              |
| <code>saga_completed / saga_compensated</code> | 1 / 1              | оба исхода Saga достигнуты                                     |
| <code>saga_duration_p95_ms</code>              | 23,325             | длительность Saga наблюдаема                                   |

## 5.4 Проверка терминологии, источников и ограничений валидности

Терминология уточнялась по уровням гарантий. At-least-once означает возможную повторную доставку сообщения. Идемпотентность означает отсутствие дополнительного побочного эффекта при повторной обработке. Однократный бизнес-результат достигается за счёт уникальных ключей, транзакционных ограничений, Inbox и дедупликации, а не за счёт абсолютной гарантии транспорта. Такая формулировка согласуется с паттерном Idempotent Receiver и описаниями *delivery guarantees* для Outbox/Inbox [9, 11, 20].

Источники проверялись по библиографическим сведениям, авторству, году публикации, URL или DOI, а также по применимости к конкретному утверждению. Дата обращения для электронных источников — 21.06.2026.

Ограничения валидности зафиксированы отдельно. Первое ограничение связано с масштабом проверки: стенд проверяет архитектурные свойства, а не поведение полного промышленного кластера с геораспределением, реальной авторизацией, платежами и миллионами WebSocket-подключений. Второе ограничение связано с измерениями: локальные задержки и пропускная способность зависят от машины, Docker и фоновой нагрузки, поэтому

используются как воспроизводимые значения конкретного стенда. Третье ограничение связано с переносом результатов: оценки сложности внедрения и стоимости эксплуатации опираются на реализованные компоненты стенда и источники; при переходе к промышленной системе они должны уточняться с учётом требований к SLA, резервированию, хранению истории и мониторингу.

## 5.5 Практические рекомендации

Практические рекомендации включались в итоговый набор только при наличии проверяемого основания. Поэтому каждая рекомендация в таблице 5.4 связана с экспериментом, метрикой или источником; рекомендации без такого основания в итоговый набор не включались.

Таблица 5.4 – Практические рекомендации по проектированию платформы

| №  | Рекомендация                                                                                                                     | Основание                                                                    | Доказательное значение                                                                                            |
|----|----------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| R1 | Хранить достоверное состояние игровой экономики, инвентаря игрока, прогресса игрока и аудита игровых операций в PostgreSQL       | серии синхронных транзакций, устаревшее чтение кэша, документация PostgreSQL | кэш вернул 100 при фактическом значении 150; критичное решение должно опираться на актуальные данные в PostgreSQL |
| R2 | Каждую внешнюю команду и callback снабжать idempotency key и фиксировать в Inbox                                                 | повторный callback, Idempotent Receiver                                      | 5 повторов дали один бизнес-результат и 4 зарегистрированных дубля                                                |
| R3 | Исходящие доменные события публиковать через Transactional Outbox и проектировать потребителей идемпотентными                    | сбой после публикации, источники по Outbox/Inbox                             | транспортный дубль появился, но outbox_pending=0 после повторной обработки                                        |
| R4 | Redis Pub/Sub использовать только для короткоживущей рассылки, а Redis Streams — для повторного чтения и контролируемой доставки | потеря Pub/Sub-сообщения, WebSocket-восстановление, документация Redis       | поздний подписчик не получил сообщение; WebSocket-восстановление получило 3 события                               |
| R5 | Saga применять только для долгих игровых экономических операций с явными компенсациями                                           | Saga-сценарий, Microsoft/InfoQ                                               | получены оба исхода: завершение и компенсация                                                                     |
| R6 | В мониторинг включить возраст очереди Outbox, отставание журнала, дубли, длительность Saga, устаревшее чтение кэша и долю ошибок | /metrics/summary, трассировка доказательств                                  | без этих метрик невозможно доказательно связать отказ с архитектурной причиной                                    |

## 6 Заключение

В ходе научно-исследовательской работы выполнены этапы 2–5 задания на практику. На этапе 2 проанализирована предметная область серверных платформ текстовых MMORPG, определены объект, предмет, цель, задачи, классы сценариев, критерии анализа и источниковая база. На этапе 3 исследованы архитектурные решения и разработан практический стенд, на котором выполнены серии 100/500/1000 операций для синхронной транзакции, Outbox-транзакции и чатовой рассылки, а также сценарии внедрения отказов: повторный callback, сбой после публикации, потеря Pub/Sub-сообщения, WebSocket-восстановление, устаревшее чтение кэша и Saga. На этапе 4 выводы верифицированы через источники, терминологию, матрицу отказов, трассировку доказательств и проверяемые метрики. На этапе 5 подготовлены письменный отчёт и приложения со вспомогательными материалами.

Цель работы достигнута: обоснована архитектурная комбинация PostgreSQL + Transactional Outbox/Inbox + Redis Streams/Pub/Sub + WebSocket-шлюз + Saga + cache-aside с разграничением областей применения. Достоверное критичное состояние игровых сущностей хранится в PostgreSQL; Outbox и Inbox обеспечивают восстанавливаемую публикацию и защиту от повторного бизнес-эффекта; Redis Pub/Sub применяется только для короткоживущей рассылки, а Redis Streams — для повторного чтения событий; WebSocket обеспечивает доставку в реальном времени при наличии механизма восстановления; Saga применяется к долгим игровым экономическим цепочкам; cache-aside ограничивается справочными и некритичными данными.

Итоговые рекомендации подтверждены экспериментальными артефактами: `summary.json`, `scenario_runs.csv`, `comparison_matrix.csv`, `evidence_trace.json`, таблицей нагрузочных серий, таблицей отказов и таблицей наблюдаемости. Проверка артефактов подтверждает наличие результатов, структуру серий, отсутствие незакрытой очереди Outbox и выполнение ключевых инвариантов.

## 7 Список использованных источников

1. *Fette I., Melnikov A.* The WebSocket Protocol. — 2011. — DOI: 10.17487/RFC6455. — URL: <https://www.rfc-editor.org/info/rfc6455> (дата обр. 21.06.2026). RFC 6455.
2. Производственная практика магистрантов: организация и проведение / Т. А. Маркина [и др.]. — Санкт-Петербург : Университет ИТМО, 2020. — 50 с. — URL: <https://books.ifmo.ru/file/pdf/2622.pdf> (дата обр. 21.06.2026).
3. *Kasenides N., Paspallis N.* A Systematic Mapping Study of MMOG Backend Architectures // Information. — 2019. — Т. 10, № 9. — DOI: 10.3390/info10090264. — URL: <https://knowledge.lancashire.ac.uk/id/eprint/29549/1/information-10-00264.pdf> (дата обр. 21.06.2026).
4. *Zhang K., Kemme B., Denault A.* Persistence in Massively Multiplayer Online Games // Proceedings of the 7th ACM SIGCOMM Workshop on Network and System Support for Games. — New York : ACM, 2008. — С. 53—58. — DOI: 10.1145/1517494.1517505. — URL: <https://www.cs.mcgill.ca/~adenau/pub/persistence.pdf> (дата обр. 21.06.2026).
5. *Zhang K., Kemme B.* Transaction Models for Massively Multiplayer Online Games // 2011 IEEE 30th International Symposium on Reliable Distributed Systems. — IEEE Computer Society, 2011. — С. 31—40. — DOI: 10.1109/SRDS.2011.13. — URL: <https://espace2.etsmtl.ca/id/eprint/16435/> (дата обр. 21.06.2026).
6. *Palant W., Griwodz C., Halvorsen P.* Consistency Requirements in Multiplayer Online Games // Proceedings of the 5th ACM SIGCOMM Workshop on Network and System Support for Games. — ACM, 2006. — С. 51. — DOI: 10.1145/1230040.1230061. — URL:



- <https://web-backend.simula.no/sites/default/files/publications/Palant.2006.2.pdf> (дата обр. 21.06.2026).
7. *Li F. W. B., Li L. W. F., Lau R. W. H.* Supporting Continuous Consistency in Multiplayer Online Games // Proceedings of the 12th Annual ACM International Conference on Multimedia. — ACM, 2004. — C. 388—391. — DOI: 10.1145/1027527.1027619. — URL: <https://frederickli.webspace.durham.ac.uk/wp-content/uploads/sites/103/2021/04/mm04-sync.pdf> (дата обр. 21.06.2026).
  8. *Diao Z.* Consistency Models for Cloud-Based Online Games: The Storage System's Perspective // 25th GI-Workshop on Foundations of Databases. — Ilmenau, 2013. — URL: [https://ceur-ws.org/Vol-1020/paper\\_03.pdf](https://ceur-ws.org/Vol-1020/paper_03.pdf) (дата обр. 21.06.2026).
  9. *Kleppmann M.* Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. — Sebastopol : O'Reilly Media, 2017. — ISBN 978-1449373320.
  10. *Hohpe G., Woolf B.* Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions. — Boston : Addison-Wesley, 2004. — ISBN 978-0321200686.
  11. *Hohpe G., Woolf B.* Idempotent Receiver / Enterprise Integration Patterns. — 2004. — URL: <https://www.enterpriseintegrationpatterns.com/patterns/messaging/IdempotentReceiver.html> (дата обр. 21.06.2026).
  12. *Hohpe G., Woolf B.* Correlation Identifier / Enterprise Integration Patterns. — 2004. — URL: <https://www.enterpriseintegrationpatterns.com/patterns/messaging/CorrelationIdentifier.html> (дата обр. 21.06.2026).
  13. *Vogels W.* Eventually Consistent // Communications of the ACM. — 2009. — T. 52, № 1. — C. 40—44. — DOI: 10.1145/1435417.1435432. — URL:

- <https://15799.courses.cs.cmu.edu/fall2013/static/papers/p40-vogels.pdf> (дата обр. 21.06.2026).
14. *Redis*. Redis Streams / Redis Documentation. — 2026. — URL: <https://redis.io/docs/latest/develop/data-types/streams/> (дата обр. 21.06.2026).
  15. *Redis*. Redis Pub/Sub / Redis Documentation. — 2026. — URL: <https://redis.io/docs/latest/develop/pubsub/> (дата обр. 21.06.2026).
  16. *PostgreSQL Global Development Group*. High Availability, Load Balancing, and Replication / PostgreSQL Documentation. — 2026. — URL: <https://www.postgresql.org/docs/current/high-availability.html> (дата обр. 21.06.2026).
  17. *PostgreSQL Global Development Group*. Table Partitioning / PostgreSQL Documentation. — 2026. — URL: <https://www.postgresql.org/docs/current/ddl-partitioning.html> (дата обр. 21.06.2026).
  18. *Microsoft Azure Architecture Center*. Event-driven architecture style / Microsoft Learn. — 07.03.2026. — URL: <https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/event-driven> (дата обр. 21.06.2026).
  19. *Richardson C*. Pattern: Transactional Outbox / Microservices.io. — 2026. — URL: <https://microservices.io/patterns/data/transactional-outbox.html> (дата обр. 21.06.2026).
  20. *Dudycz O*. Outbox, Inbox patterns and delivery guarantees explained / Event-Driven.io. — 30.12.2020. — URL: [https://event-driven.io/en/outbox\\_inbox\\_patterns\\_and\\_delivery\\_guarantees\\_explained/](https://event-driven.io/en/outbox_inbox_patterns_and_delivery_guarantees_explained/) (дата обр. 21.06.2026).
  21. *Microsoft Azure Architecture Center*. Saga distributed transactions pattern / Microsoft Learn. — 2026. — URL: <https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/saga>

- us/azure/architecture/patterns/saga (дата обр. 21.06.2026).
22. *Morling G.* Saga Orchestration for Microservices Using the Outbox Pattern / InfoQ. — 25.02.2021. — URL: <https://www.infoq.com/articles/saga-orchestration-outbox/> (дата обр. 21.06.2026).
  23. *Ably Realtime.* WebSocket architecture best practices: Designing scalable realtime systems / Ably. — 05.11.2024. — URL: <https://ably.com/topic/websocket-architecture-best-practices> (дата обр. 21.06.2026).
  24. *O’Riordan M.* WebSockets at Scale: Architecture for Millions of Connections / WebSocket.org. — 02.09.2024. — URL: <https://websocket.org/guides/websockets-at-scale/> (дата обр. 21.06.2026) ; Обновлено 10.03.2026.
  25. *Alpatov A. N., Iurov I. I.* Algorithm and Software Implementation of Real-Time Collaborative Editing of Graphical Schemes Using Socket.IO Library // Программные системы и вычислительные методы. — 2024. — № 1. — С. 10—19. — DOI: 10.7256/2454-0714.2024.1.70173. — URL: <https://journals.rcsi.science/2454-0714/article/view/359422> (дата обр. 21.06.2026).
  26. *Fedulov A. A.* Designing the Architecture of the Client-Server Interaction Protocol for Web Applications Based on WebSocket // Программные системы и вычислительные методы. — 2025. — № 3. — С. 20—30. — DOI: 10.7256/2454-0714.2025.3.75392. — URL: <https://journals.rcsi.science/2454-0714/article/view/359337> (дата обр. 21.06.2026).
  27. *Горинов Д. А.* MMORPG Architecture Research Stand. — 2026. — URL: <https://github.com/gorinovdan/mmorpg-architecture-research> (дата обр. 21.06.2026) ; Исследовательский стенд для проверки Outbox/Inbox, Redis Streams/Pub/Sub, WebSocket reconnect, Saga и cache-aside.

28. *Tessier J.* Why your caching strategies might be holding you back (and what to consider next) / Redis Developer Blog. — 13.06.2025. — URL: <https://redis.io/blog/why-your-caching-strategies-might-be-holding-you-back-and-what-to-consider-next/> (дата обр. 21.06.2026).

## Репозиторий, команды запуска и структура стенда

Публичный репозиторий исследовательского стенда:

[github.com/gorinovdan/mmorpg-architecture-research](https://github.com/gorinovdan/mmorpg-architecture-research)

### Команды запуска и проверки стенда

| Команда                          | Назначение                                                                  |
|----------------------------------|-----------------------------------------------------------------------------|
| <code>make deps</code>           | Создать Python-окружение и установить зависимости для графиков.             |
| <code>make test</code>           | Запустить Go-тесты, проверить Docker Compose и Python-скрипты.              |
| <code>make up</code>             | Собрать и поднять PostgreSQL, Redis, серверную часть и фоновые обработчики. |
| <code>make migrate</code>        | Проверить применение миграций серверной части при старте.                   |
| <code>make experiment</code>     | Выполнить серии нагрузки и сценарии внедрения отказов.                      |
| <code>make verify-results</code> | Проверить инварианты, матрицы результатов и трассировку доказательств.      |
| <code>make down</code>           | Остановить контейнеры стенда.                                               |

### Структура стенда

| Путь                                    | Назначение                                                                                                       |
|-----------------------------------------|------------------------------------------------------------------------------------------------------------------|
| <code>cmd/researchd</code>              | Серверная часть и фоновые обработчики: HTTP API, WebSocket, PostgreSQL, Redis, Outbox, Inbox, Saga, cache-aside. |
| <code>cmd/wscheck</code>                | Проверка WebSocket-восстановления после подключения.                                                             |
| <code>internal/expstats</code>          | Проверяемые функции расчёта процентилей и дублей.                                                                |
| <code>scripts/run_experiments.py</code> | Запуск сценариев нагрузки и отказов, сбор CSV/JSON и вспомогательных графиков.                                   |
| <code>scripts/verify_results.py</code>  | Проверка инвариантов, артефактов и трассировки доказательств.                                                    |
| <code>docs/architecture.md</code>       | Описание границ стенда, компонентов, потоков данных и метрик.                                                    |
| <code>docs/experiments.md</code>        | Методика экспериментов, артефакты и правила интерпретации.                                                       |
| <code>.github/workflows/ci.yml</code>   | GitHub Actions для Go-тестов, Docker smoke test и проверки артефактов.                                           |

### Raw-сводка экспериментальных артефактов

Приложение содержит сокращённую сводку исходных артефактов стенда. Интерпретация результатов приведена в основной части отчёта; здесь зафиксированы файлы, по которым проверяется воспроизводимость.

Файлы результатов `results/latest`

| Файл                               | Содержание                                                                                                                                        |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>summary.json</code>          | Сводка запуска: seed, серии нагрузки, сценарии внедрения отказов, финальные метрики, матрица сравнения, трассировка доказательств и рекомендации. |
| <code>metrics.csv</code>           | Нормализованная выгрузка метрик в формате <code>scenario, metric, value, unit</code> .                                                            |
| <code>scenario_runs.csv</code>     | 45 отдельных запусков: 3 сценария, 3 размера нагрузки, 5 повторов.                                                                                |
| <code>comparison_matrix.csv</code> | Экспериментальная матрица решений по критериям.                                                                                                   |
| <code>criteria_scores.csv</code>   | Развёрнутая форма оценок по критериям для проверки итоговой матрицы.                                                                              |
| <code>evidence_trace.json</code>   | Машиночитаемая цепочка «критерий — сценарий — метрика — результат — вывод».                                                                       |
| <code>recommendations.json</code>  | Рекомендации с указанием экспериментального основания.                                                                                            |

Сокращённая raw-сводка `summary.json`

| Блок                     | Поле                                           | Значение              |
|--------------------------|------------------------------------------------|-----------------------|
| Параметры запуска        | seed; размеры нагрузки; повторы                | 4116; 100/500/1000; 5 |
| Объём серий              | запуски сценариев                              | 45                    |
| Чат                      | сообщения; длина журнала                       | 8000; 8000            |
| Inbox/Outbox             | записей Inbox; опубликовано; попытки; остаток  | 16003; 8004; 8005; 0  |
| Redis Streams            | длина журнала; дубли событий                   | 8005; 1               |
| Pub/Sub                  | доставка позднему подписчику                   | false                 |
| Cache-aside              | значение в кэше; актуальное значение; устарело | 100; 150; true        |
| Saga                     | завершено; компенсировано; p95 длительности    | 1; 1; 23,325 мс       |
| WebSocket-восстановление | получено                                       | 3                     |

## Протокол проверки источников и оформления

### Протокол проверки отчёта, источников и стенда

| Проверка                           | Выполненное действие                                                                                                                                                           | Результат     |
|------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|
| Структура по методическому пособию | Проверены титульный лист, содержание, список сокращений, введение, основная часть, заключение, источники и приложения                                                          | Соответствует |
| Оформление                         | Проверены формат А4, поля 30/15/20/20 мм, Times New Roman 14 пт, полуторный интервал, ссылки в квадратных скобках и расположение приложений после источников                   | Соответствует |
| Источники                          | Проверены библиографические сведения, URL/DOI, применимость утверждений и дата обращения 21.06.2026                                                                            | Подтверждено  |
| Стенд                              | Выполнены <code>make deps</code> , <code>make test</code> , <code>make up</code> , <code>make migrate</code> , <code>make experiment</code> , <code>make verify-results</code> | Успешно       |
| Детерминированность структуры      | Эксперимент повторён с seed 4116; проверены структура артефактов и инварианты, абсолютные значения задержки не фиксировались как неизменные                                    | Подтверждено  |
| Визуальная проверка                | Проверены титульный лист, содержание, план-график, методика, таблицы, рисунки, заключение и начало приложений                                                                  | Выполнено     |
| GitHub                             | Проверена доступность репозитория, публикация обновлённых артефактов и последний запуск GitHub Actions                                                                         | Подтверждено  |